MovieLens Project Report

Fabian Hiernaux

2026-05-02

Contents

1 Introduction

The goal of this project is to develop a movie recommendation system using the MovieLens 10M dataset. In a world of overflowing digital content, recommendation algorithms are essential tools that help users discover items they are likely to enjoy based on their previous behavior and the behavior of others. The main objective is to develop an algorithm capable of predicting user ratings with a high degree of accuracy, as measured by the root mean square error (RMSE).

1.1 Project Objective

The primary mission is to build a machine learning model capable of predicting movie ratings with high precision. Accuracy is measured by the Root Mean Square Error (RMSE). Our target is to achieve a final RMSE lower than 0.86490.

1.2 Environnement et contraintes techniques

A significant aspect of this work is the technical constraint under which it was performed. The analysis was conducted on a Virtual Machine (based on virtualbox) limited to 6 GB of RAM and 4CPUs.

This constraint influenced the modeling strategy:

- Efficiency: We prioritized a regularized linear model optimised by backfitting over more memory-intensive techniques. This approach offers an excellent balance between predictive performance and resource consumption, whereas more resource-intensive methods (such as complex matrix factorisation) might have overloaded the system.
- Optimization: Explicit memory management (using gc() to clean up memory) was integrated into the workflow to ensure the system remained stable despite the large dataset size (10 million entries).

2 Methods and Analysis

2.1 Data Preparation

As per the instructions provided by the HarvardX course, the initial code to download the MovieLens 10M dataset and split it into two sets was used:

1. **edx set**: Used for data exploration, model training, and parameter tuning.
2. **final_holdout_test set**: Kept strictly separate and used only for the final assessment of the model's accuracy.

```
# Note: this process could take a couple of minutes

if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us.r-project.org")
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")

library(tidyverse)
library(caret)

# MovieLens 10M dataset:
# https://grouplens.org/datasets/movielens/10m/
# http://files.grouplens.org/datasets/movielens/ml-10m.zip

options(timeout = 120)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::"), simplify = TRUE),
                         stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
```

```
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::"), simplify = TRUE),
                        stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
movies <- movies %>%
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

# Final hold-out test set will be 10% of MovieLens data
set.seed(1, sample.kind="Rounding") # if using R 3.6 or later
# set.seed(1) # if using R 3.5 or earlier
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

# Make sure userId and movieId in final hold-out test set are also in edx set
final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

# Add rows removed from final hold-out test set back into edx set
removed <- anti_join(temp, final_holdout_test)
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

2.1.1 Feature Engineering

To move beyond a simple average, we extracted two key temporal variables from the raw data: - **Release Year**: Extracted from the movie title to see if "classics" are rated differently than modern blockbusters. - **Review Week (Date)**: Extracted from the timestamp to capture trends in rating behavior over time.

Before proceeding with the exploratory analysis, we transform the raw data to extract specific features. This step is essential because our final modeling strategy (detailed in Section 4) will move beyond simple averages to incorporate temporal and movie-age effects. By extracting the release year from the titles and converting the timestamps into weekly dates at this stage, we can visualize these trends before formally integrating them into our regularized model.

```
# Extracting release year and converting timestamp to weekly dates
# This ensures that these columns are available for both EDA and modeling

edx <- edx %>%
  mutate(release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"), "\\d{4}")),
         date = round_date(as_datetime(timestamp), unit = "week"))

final_holdout_test <- final_holdout_test %>%
  mutate(release_year = as.numeric(str_extract(str_extract(title, "\\(\\d{4}\\)$"), "\\d{4}")),
         date = round_date(as_datetime(timestamp), unit = "week"))
```

2.1.2 Data Inspection & Computational Optimization

Before building the model, we performed a rigorous inspection of the `edx` dataset to validate its structure and content. This initial exploration validates the data structure and answers the project's fundamental questions through direct code execution.

In summary, the key questions and their answers are as follows:

- **Dataset Dimensions**: The dataset contains **9,000,055 rows** and **6 columns**.
- **Rating Values**:
  - No ratings of **0** were given, which is consistent with the MovieLens scale (0.5 to 5).
  - There are **2,121,240** ratings of **3**.
- **Unique Entries**: There are **10,677 unique movies** and **69,878 unique users**.
- **Genre Distribution**: Drama is the most rated genre (**3,910,127**), followed by Comedy (**3,540,930**), Thriller (**2,325,899**), and Romance (**1,712,100**).
- **Most Popular Movie**: *Pulp Fiction (1994)* holds the record for the greatest number of ratings.
- **Rating Frequency**: The five most common ratings are, in order: **4, 3, 5, 3.5, and 2**.
- **Rating Type Bias**: As observed, half-star ratings are significantly less common than whole-star ratings, suggesting a psychological preference for integer scores.

2.1.2.1 Dataset Dimensions and Basic Statistics

We first verified the volume of data and the range of ratings.

```
# 1. Structure and dimensions of edx
res_dim <- dim(edx)
message(paste("The edx dataset has", res_dim[1], "rows and", res_dim[2], "columns."))

# 2. Preview of the data
head(edx)
```

```
##   userId movieId rating timestamp                         title
## 1      1     122      5 838985046              Boomerang (1992)
## 2      1     185      5 838983525               Net, The (1995)
## 4      1     292      5 838983421               Outbreak (1995)
## 5      1     316      5 838983392               Stargate (1994)
## 6      1     329      5 838983392 Star Trek: Generations (1994)
## 7      1     355      5 838984474       Flintstones, The (1994)
##                           genres release_year       date
## 1                Comedy|Romance         1992 1996-08-04
## 2          Action|Crime|Thriller         1995 1996-08-04
## 4  Action|Drama|Sci-Fi|Thriller         1995 1996-08-04
## 5       Action|Adventure|Sci-Fi         1994 1996-08-04
## 6 Action|Adventure|Drama|Sci-Fi         1994 1996-08-04
## 7       Children|Comedy|Fantasy         1994 1996-08-04
```

```
# 3. Summary of key statistics
summary(edx)
```

```
##      userId         movieId          rating        timestamp
##  Min.   :    1   Min.   :    1   Min.   :0.500   Min.   :7.897e+08
##  1st Qu.:18124   1st Qu.:  648   1st Qu.:3.000   1st Qu.:9.468e+08
```

```
##  Median :35738   Median : 1834   Median :4.000   Median :1.035e+09
##  Mean   :35870   Mean   : 4122   Mean   :3.512   Mean   :1.033e+09
##  3rd Qu.:53607   3rd Qu.: 3626   3rd Qu.:4.000   3rd Qu.:1.127e+09
##  Max.   :71567   Max.   :65133   Max.   :5.000   Max.   :1.231e+09
##     title              genres          release_year
##  Length:9000055     Length:9000055     Min.   :1915
##  Class :character   Class :character   1st Qu.:1987
##  Mode  :character   Mode  :character   Median :1994
##                                        Mean   :1990
##                                        3rd Qu.:1998
##                                        Max.   :2008
##       date
##  Min.   :1995-01-08 00:00:00.00
##  1st Qu.:2000-01-02 00:00:00.00
##  Median :2002-10-27 00:00:00.00
##  Mean   :2002-09-21 12:42:48.57
##  3rd Qu.:2005-09-18 00:00:00.00
##  Max.   :2009-01-04 00:00:00.00
```

```r
# 4. Counting unique users and movies, numbers of zeros and threes
n_unique_users <- n_distinct(edx$userId)
n_unique_movies <- n_distinct(edx$movieId)

message(paste("There are", n_unique_users, "unique users and", n_unique_movies, "unique movies in edx."))

# Verification of specific ratings. Using efficient vectorized sums for specific counts
count_0 <- sum(edx$rating == 0)
count_3 <- sum(edx$rating == 3)
# Insight: In the MovieLens 10M dataset, ratings are on a 0.5 to 5.0 scale.
# A count of 0 for the "0" rating confirms the expected range.
message(paste("Number of 0 ratings:", count_0))
message(paste("Number of 3 ratings:", count_3))
```

The inspection of the `edx` dataset reveals a massive volume of data, consisting of **9,000,055** rows and **8** columns.

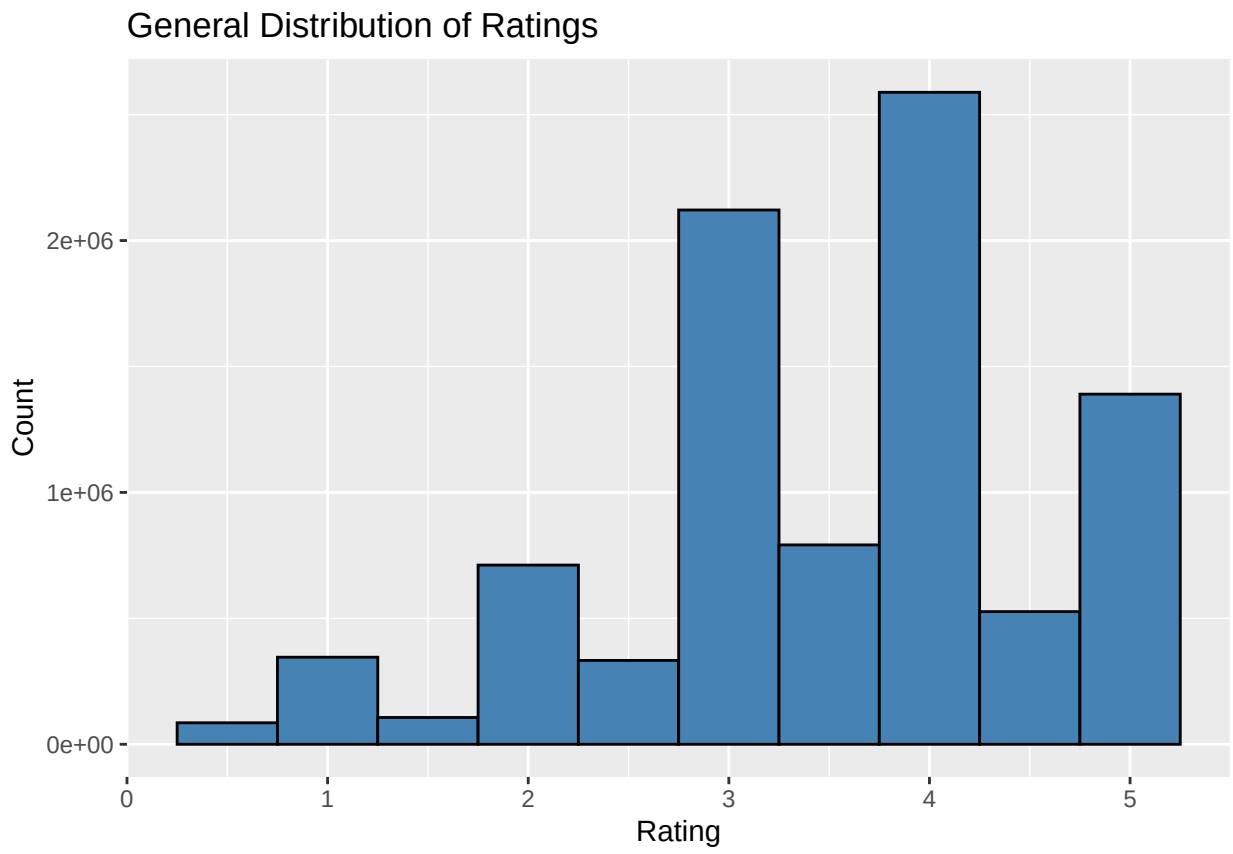
Among these records, we identify: - **69,878** unique users. - **10,677** unique movies.

Regarding the rating values, we confirmed the integrity of the scale: there are **0** ratings of zero (as expected for a 0.5 to 5.0 scale), while the score of 3 appears **2,121,240** times.

2.1.2.2 Visualizing the Rating Distribution

The following visualization helps determine if users tend to be generous or strict, and highlights the preference for specific types of scores.

How do people rate movies?



The histogram reveals a "positivity bias," with most ratings above 3. Furthermore, we can clearly see that whole-star ratings are more frequent than half-star ratings.

2.1.2.3 Overcoming Memory Constraints: Genre Analysis One of the main technical challenges of this project was the 6 GB RAM limitation of the Virtual Machine. A standard Tidyverse approach to count genres (using separate_rows() on 9 million rows) causes system instability due to memory exhaustion.

To identify the top genres, we evaluated three different algorithms to balance accuracy and performance:

- Method 1 (Sampling): Taking a 1M row sample for quick estimation.
- Method 2 (String Detection): Using str_detect for targeted counting.
- Method 3 (Pre-aggregation): The most efficient approach. By grouping the data by unique genre combinations before expanding them, we reduced the workload from 9 million rows to only ~800 unique strings.

We chose Method 3 for its mathematical identity to the standard approach while maintaining a minimal memory footprint.

```
# Optimal Engineering approach (Method 3)
top_genres_m3 <- edx %>%
  group_by(genres) %>%
  summarize(n = n()) %>%
  separate_rows(genres, sep = "\\|") %>%
  group_by(genres) %>%
  summarize(count = sum(n)) %>%
```

```
  arrange(desc(count))

head(top_genres_m3, 10)
```

```
## # A tibble: 10 x 2
##    genres      count
##    <chr>       <int>
##  1 Drama     3910127
##  2 Comedy    3540930
##  3 Action    2560545
##  4 Thriller  2325899
##  5 Adventure 1908892
##  6 Romance   1712100
##  7 Sci-Fi    1341183
##  8 Crime     1327715
##  9 Fantasy    925637
## 10 Children   737994
```

The analysis confirms that Drama is the most rated genre, followed by Comedy and Thriller.

2.1.2.4 Specific Findings Finally, we identified the most popular items and the frequency of specific scores to confirm our modeling assumptions.

```
# Most rated movie
edx %>%
  group_by(movieId, title) %>%
  summarize(count = n()) %>%
  arrange(desc(count)) %>%
  head(1)
```

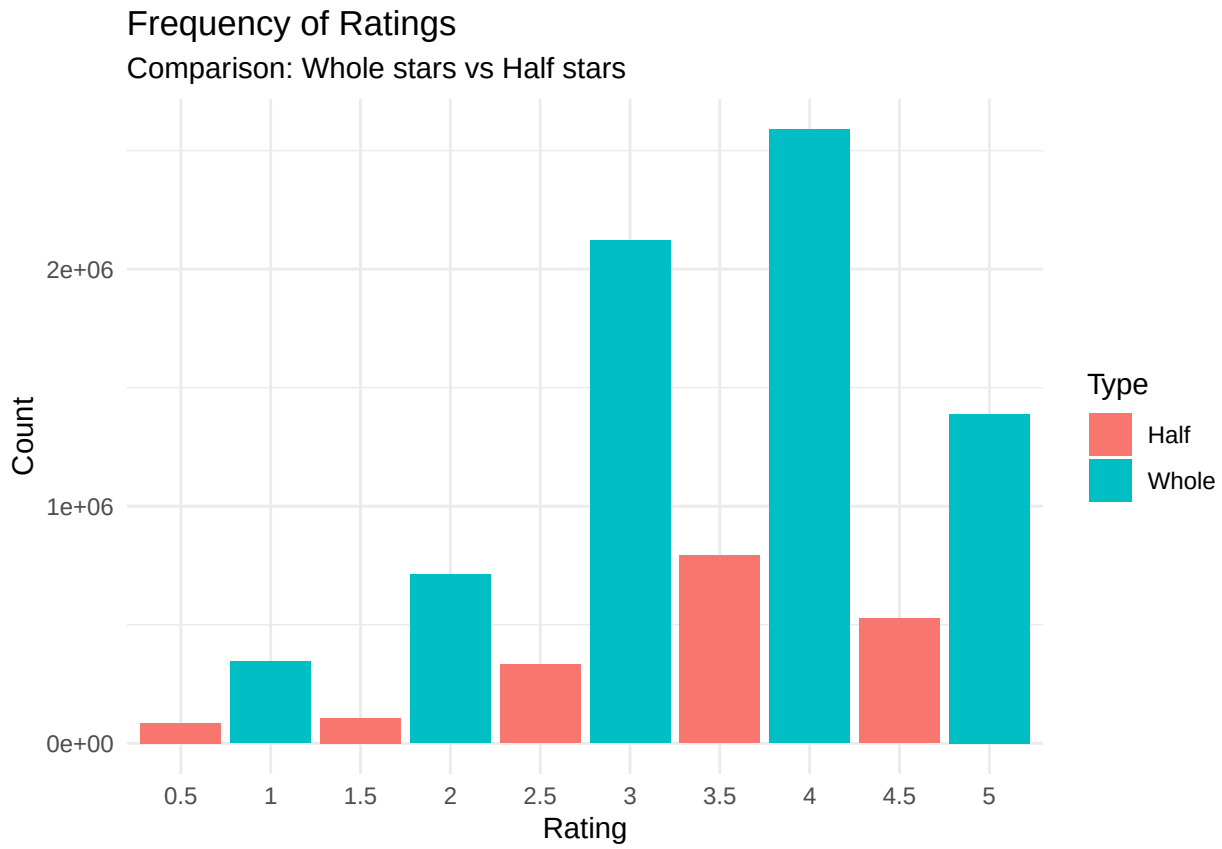
```
## # A tibble: 1 x 3
## # Groups:   movieId [1]
##   movieId title                count
##     <int> <chr>                <int>
## 1     296 Pulp Fiction (1994) 31362
```

```
# Top 5 most given ratings
edx %>%
  group_by(rating) %>%
  summarize(count = n()) %>%
  arrange(desc(count)) %>%
  head(5)
```

```
## # A tibble: 5 x 2
##   rating   count
##    <dbl>   <int>
## 1    4   2588430
## 2    3   2121240
## 3    5   1390114
## 4    3.5  791624
## 5    2    711422
```

2.1.2.5 Psychological Bias: Whole vs. Half Stars The visualization below reinforces the observation that users have a psychological preference for integer ratings.

```
edx %>%
  group_by(rating) %>%
  summarize(count = n()) %>%
  mutate(half_star = ifelse(rating %% 1 == 0, "Whole", "Half")) %>%
  ggplot(aes(x = factor(rating), y = count, fill = half_star)) +
  geom_bar(stat = "identity") +
  labs(title = "Frequency of Ratings",
       subtitle = "Comparison: Whole stars vs Half stars",
       x = "Rating", y = "Count", fill = "Type") +
  theme_minimal()
```

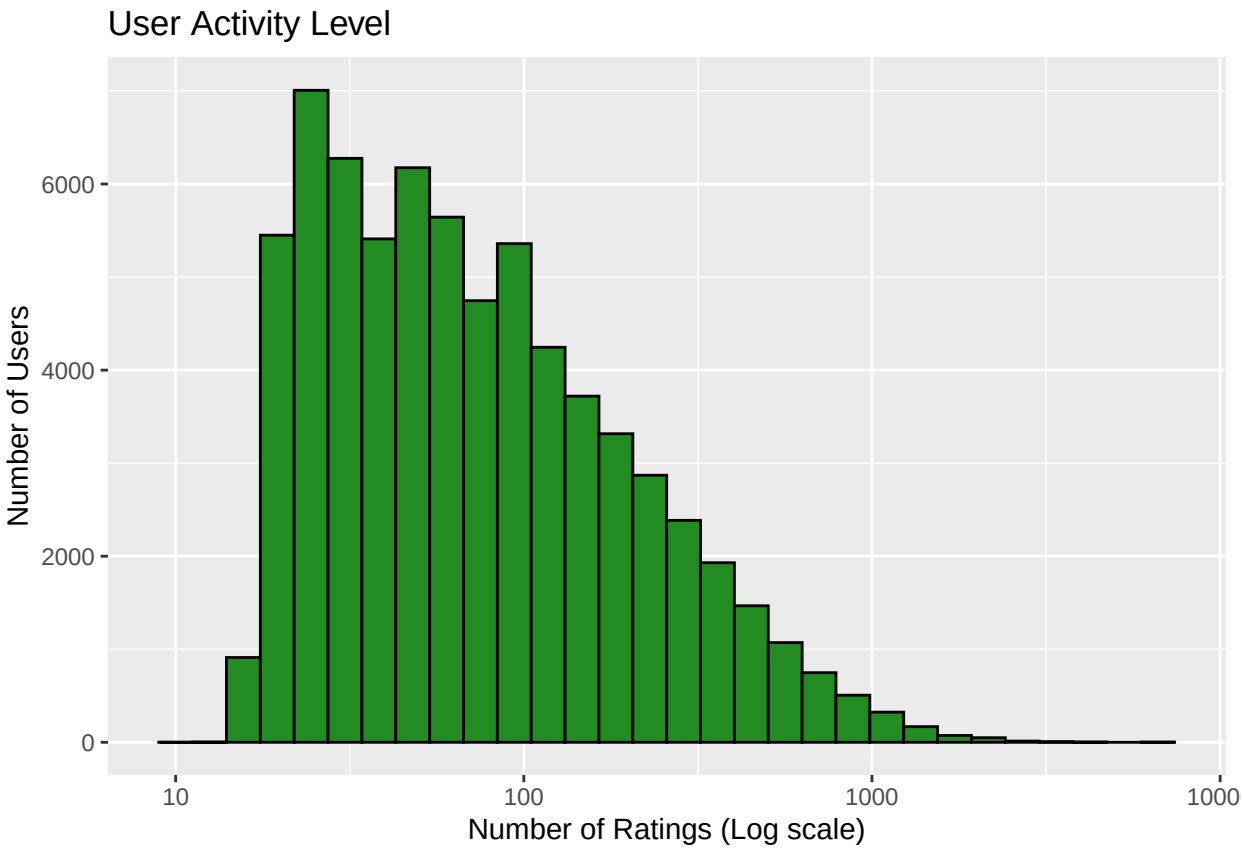


2.2 Exploratory Data Analysis (EDA)

Before building the model, we must understand the "signals" in our data.

2.2.1 The User Effect

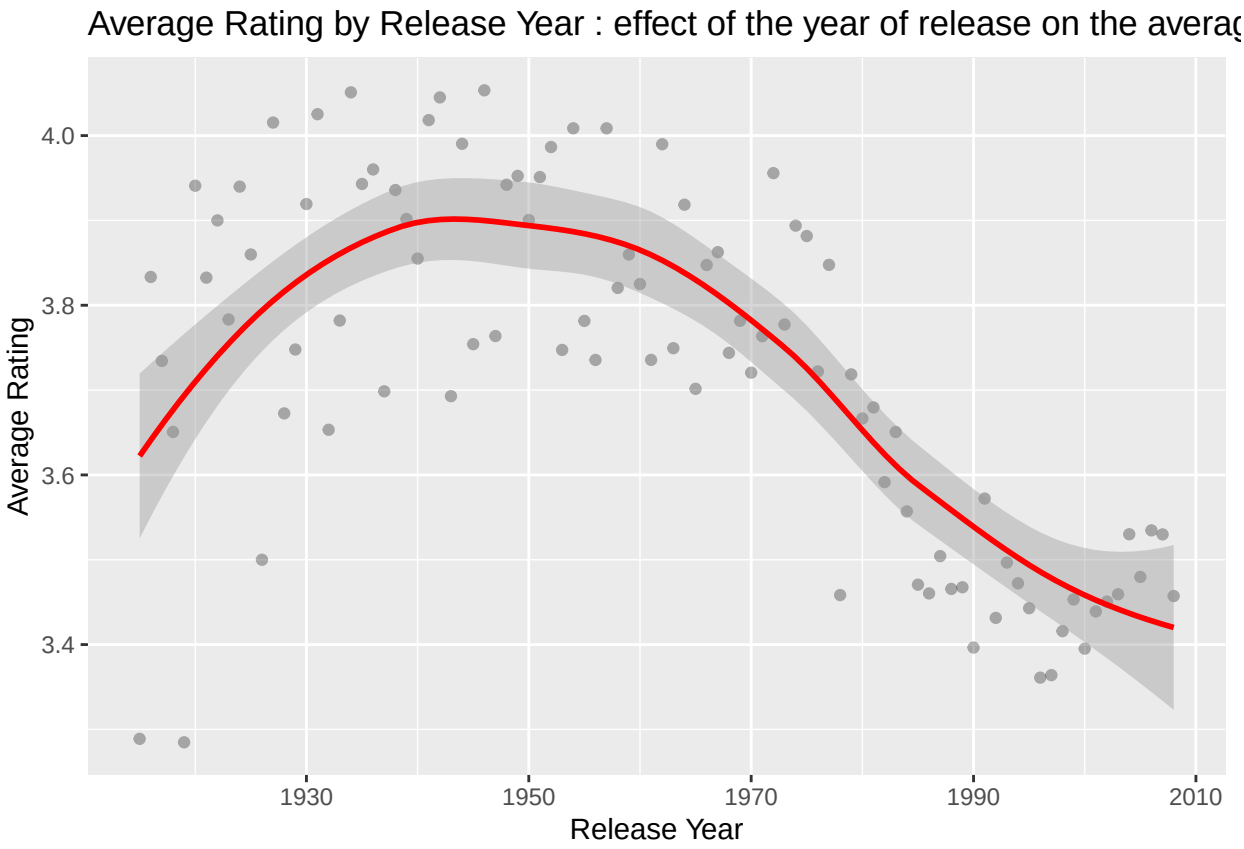
Some users are critical, while others are generous. Some have rated hundreds of movies, while others have rated only a few.



This diversity is why we need Regularization. If a user has only rated one movie, we shouldn't trust their "bias" as much as a user who has rated five hundred. Regularization "penalizes" estimates based on small sample sizes to prevent errors.

2.2.2 The Movie Release Year Effect

Does the age of a movie affect its rating?



We observe that older movies (classics) tend to have higher average ratings than modern ones. This confirms that the Release Year is a significant predictor to include in our model.

‘‘‘